IT5082 – Optimization Methods
Programming Assignment Report

Capacitated Facility Location Problem using the Olist Brazilian E-Commerce Dataset

MS26916576 – Malagala P R
MS26915968 – Kumara I P S

MSc in Artificial Intelligence – SLIIT | October 2026

GitHub: https://github.com/SajithKumara/IT-5082-OM-Assignment

Abstract

This report presents a group programming assignment for the IT5082 Optimization Methods module. The problem selected is the Capacitated Facility Location Problem (CFLP), modelled using real-world logistics data from the Olist Brazilian e-commerce dataset. The objective is to determine which candidate warehouse locations to open and which customer cities each warehouse should serve, minimising combined fixed opening costs and transport costs subject to capacity constraints. Two methods are implemented and compared: an exact method using Mixed Integer Linear Programming (MILP) solved with PuLP, and a heuristic method using Greedy construction followed by Simulated Annealing (SA). The MILP finds provably optimal solutions on all four instance sizes but runtime grows sharply with problem size. SA scales better in runtime but produces solutions with an optimality gap that increases with problem size, from 0% on the smallest instance to approximately 33% on the largest. The results demonstrate the classic trade-off between exactness and scalability in NP-hard combinatorial optimization.

1. Problem Statement

Olist is a Brazilian e-commerce marketplace that connects small and medium-sized businesses to customers across Brazil. Merchants sell through the Olist Store and ship directly to customers using logistics partners. As the company serves customers distributed across a geographically large country, a key operational decision is where to locate regional warehouses to minimise total logistics costs.

This assignment frames the problem as a Capacitated Facility Location Problem (CFLP): given a set of candidate warehouse sites and customer cities, decide which warehouses to open and which customers each warehouse should serve. Each warehouse has a fixed opening cost and a capacity limit. Each customer must be assigned to exactly one warehouse (single-sourcing). The goal is to minimise the sum of fixed opening costs and transport costs.

CFLP is NP-hard; exact methods guarantee optimal solutions but face exponential worst-case runtime as problem size grows. This assignment compares an exact MILP formulation with a Simulated Annealing heuristic to demonstrate this fundamental trade-off.

2. Dataset Description

Source: Olist Brazilian E-Commerce Public Dataset, Kaggle.
https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

The dataset contains approximately 100,000 anonymised orders placed between 2016 and 2018 across multiple Brazilian marketplaces. Three files were used: olist_customers_dataset.csv (customer city and zip code), olist_orders_dataset.csv (delivery status), and olist_geolocation_dataset.csv (zip code to latitude/longitude mapping).

2.1 Instance Construction

The preprocessing script aggregates delivered orders by city-state pair to compute demand, computes median coordinates per city, and builds a haversine distance matrix. Four instances of increasing size were created:

Instance	Customer cities (n)	Candidate sites (m)	Total demand (orders)
cflp_n20_m10	20	10	~44,680
cflp_n50_m15	50	15	~76,035
cflp_n100_m25	100	25	~89,208
cflp_n200_m40	200	40	~97,038

Table 1: CFLP instances built from Olist data.

2.2 Modelling Assumptions

Three parameters are not present in the raw data and are assumed with justification:

Transport cost: 1 cost unit per order per km, making costs proportional to both distance and volume.

Opening costs: Scaled to match the magnitude of transport costs; larger cities up to 25% more expensive, reflecting land and infrastructure prices.

Capacity: Each warehouse handles at most 30% of total demand, requiring at least four open warehouses and creating a non-trivial allocation problem.

3. Mathematical Formulation

3.1 Sets and Parameters

$i \in \{1,\ldots,m\}$ – candidate warehouse locations; $j \in \{1,\ldots,n\}$ – customer cities

f_i – fixed opening cost of warehouse i; c_{ij} – cost of serving customer j from warehouse i (distance × demand)

d_j – demand of customer city j; C_i – capacity of warehouse i

3.2 Decision Variables

$y_i \in \{0,1\}$ – 1 if warehouse i is opened, 0 otherwise

$x_{ij} \in \{0,1\}$ – 1 if customer j is assigned to warehouse i, 0 otherwise

3.3 Objective Function

Minimise total fixed opening costs plus total transport costs:

$$min \ \Sigma_i f_i y_i + \Sigma_i \Sigma_j c_{ij} x_{ij}$$

3.4 Constraints

C1 – Every customer served by exactly one warehouse: $\Sigma_i x_{ij} = 1$ for all j. Each customer city must be assigned to exactly one warehouse, not zero and not more than one.

C2 – Only assign to open warehouses: $x_{ij} \leq y_i$ for all i, j. A customer cannot be assigned to a warehouse that is not open.

C3 – Capacity not exceeded: $\Sigma_j d_j x_{ij} \leq C_i y_i$ for all i. Total demand handled by any warehouse must not exceed its capacity. When $y_i = 0$ the right side becomes zero, which also enforces C2, making C2 technically redundant but retained for clarity and solver performance.

C4 – Binary variables: $x_{ij}, y_i \in \{0,1\}$.

4. Algorithms and Justification

4.1 Mixed Integer Linear Programming (Exact Method)

Mixed Integer Linear Programming (MILP) is an exact optimization method that finds the globally optimal solution by searching the full feasible space. The standard approach for binary variables is Branch and Bound: the LP relaxation (allowing fractional variable values) is solved first to obtain a lower bound on the optimal integer solution. The solver then branches on fractional variables, adding integrality constraints at each node. When the LP relaxation bound at a node equals the best known integer solution, optimality is proved and the search terminates. Branches whose lower bound exceeds the best known integer cost are pruned, reducing the search space. This LP relaxation lower bound is the basis for the optimality gap reported in the results.

MILP was selected because the CFLP formulation maps directly to a binary ILP with warehouse and assignment variables, as covered in Lecture 5. The PuLP library was used with the open-source CBC solver and a 300-second time limit per instance. The key limitation is that computation time grows rapidly with problem size due to the NP-hard nature of integer programming.

4.2 Greedy Construction + Simulated Annealing (Heuristic Method)

Simulated Annealing (SA) is a metaheuristic inspired by the physical annealing process in metallurgy, as covered in Lecture 10. A high initial temperature allows the search to accept worse solutions with probability $P = \exp(-\Delta E/T)$, enabling escape from local optima. As temperature is reduced by the cooling schedule $T \leftarrow T \times \alpha$, the search becomes increasingly selective and converges to a high-quality solution.

The implementation uses two phases. First, a greedy initialisation assigns customers in descending demand order, each to the lowest-cost feasible warehouse, producing a valid starting solution quickly. Second, SA improves this solution using five neighbourhood moves: reassign (move one customer to a different warehouse), customer_swap

(exchange two customers' assignments), open (open a new warehouse and move a customer to it), close (close one warehouse and redistribute its customers), and facility_swap (replace one open warehouse with a closed one). All neighbourhood moves always return feasible solutions, so no penalty functions are needed. Parameters: 2,000 iterations per run, cooling rate α = 0.995, initial temperature T_0 = max(1, 0.01 × greedy cost), 10 independent seeds per instance.

5. Experimental Setup

Four instances from Olist data: n = 20, 50, 100, 200 customer cities.

Both methods use the same evaluate(solution, instance) function defined in cflp_common.py, ensuring identical cost computation and feasibility checking for a fair comparison.

MILP: PuLP 2.9.0 with CBC solver, 300-second time limit per instance. The CBC solver reports a None gap when it closes the optimality gap to zero before logging a separable bound; this is treated as 0% gap throughout the results.

SA: 10 seeds per instance (seeds 0–9), 2,000 iterations per run, cooling rate 0.995. Runtime includes both greedy initialisation and SA search.

Comparison metrics: solution cost, optimality gap (%), runtime (seconds), scalability across instance sizes, and solution variance across seeds.

Environment: Google Colab, Python 3. Libraries: PuLP 2.9.0, NumPy, Pandas, Matplotlib.

6. Results

6.1 MILP Results

The MILP solver found a provably optimal solution for all four instances within the 300-second time limit. The CBC solver closed the optimality gap to zero in every case, confirming global optimality. The raw notebook output reports the gap as None when the solver converges before logging a separable LP bound; this is equivalent to a 0% gap.

Instance	Customers	Sites	Status	Cost	Gap (%)	Runtime (s)	Open sites
cflp_n20_m10	20	10	Optimal	10,831,298	0.0	0.03	5
cflp_n50_m15	50	15	Optimal	13,876,029	0.0	0.25	6
cflp_n100_m25	100	25	Optimal	14,145,513	0.0	3.27	8
cflp_n200_m40	200	40	Optimal	13,844,672	0.0	39.15	10

Table 2: MILP results across all four instances.

Runtime grows from 0.03s at n=20 to 39.15s at n=200, demonstrating the scalability challenge of exact methods. The solver opened 5–10 warehouses per instance, reflecting

the 30% capacity constraint. The n=200 optimal cost (13,844,672) is slightly lower than n=100 (14,145,513) because the larger instance provides 40 candidate sites instead of 25, giving the solver more flexibility to find cheaper warehouse assignments. On the 200-customer instance, the optimal solution placed warehouses across Brazil's main regions: São Paulo state (Jundiaí, Mauá), Rio de Janeiro (São Gonçalo), South (Curitiba, Porto Alegre), Northeast (Salvador, Recife, São Luís), Centre-West (Goiânia), and Minas Gerais (Contagem).

6.2 Simulated Annealing Results

SA was run with 10 seeds per instance. All 40 runs completed successfully and produced feasible solutions. Runtimes reported below are the total per-run time including both greedy initialisation and SA search.

Instance	Greedy cost	Best SA cost	Mean SA cost	Std	Best gap (%)	Mean gap (%)	Mean total time (s)
cflp_n20_m10	14,439,731	10,831,298	11,449,238	491,853	0.00	5.71	5.91
cflp_n50_m15	22,372,209	15,309,570	16,414,763	1,120,896	10.33	18.30	16.16
cflp_n100_m25	23,779,245	16,208,016	17,036,126	927,343	14.58	20.43	45.96
cflp_n200_m40	20,067,629	18,352,860	19,238,272	714,309	32.56	38.96	130.31

Table 3: SA results (10 seeds per instance). Gap is relative to MILP optimal cost. Runtime = greedy initialisation + SA search.

SA achieves the MILP optimum on n=20 (best gap 0%). The gap grows with problem size: 10.3% at n=50, 14.6% at n=100, and 32.6% at n=200. SA substantially reduces cost from the greedy starting solution in all cases — on n=100 it improved the greedy cost of 23,779,245 by approximately 32% to reach a best of 16,208,016.

6.3 Method Comparison

Instance	MILP cost	Best SA	Mean SA	Best gap (%)	Mean gap (%)	MILP time (s)	SA total time (s)
cflp_n20_m10	10,831,298	10,831,298	11,449,238	0.00	5.71	0.03	5.91
cflp_n50_m15	13,876,029	15,309,570	16,414,763	10.33	18.30	0.25	16.16
cflp_n100_m25	14,145,513	16,208,016	17,036,126	14.58	20.43	3.27	45.96
cflp_n200_m40	13,844,672	18,352,860	19,238,272	32.56	38.96	39.15	130.31

Table 4: Direct comparison of MILP and SA. MILP time is solve time only; SA total time includes greedy initialisation and SA search.

MILP is faster than SA on small instances (0.03s vs 5.91s at n=20) because the CBC solver resolves small problems almost instantly. At n=200, MILP takes 39.15s while SA averages 130.31s total per run. However, SA runtime is fully controllable via the iteration count and can be reduced for larger instances, while MILP growth is problem-dependent and can exceed any fixed time limit on hard instances.

6.4 Plots

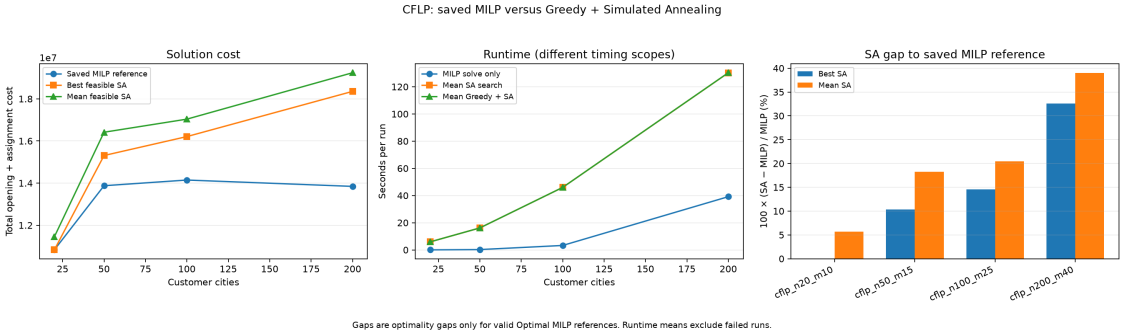


Figure 1: Left – solution cost (MILP reference, best SA, mean SA). Centre – runtime comparison (note different timing scopes: MILP is solve only, SA is greedy + search). Right – SA optimality gap (%) by instance.

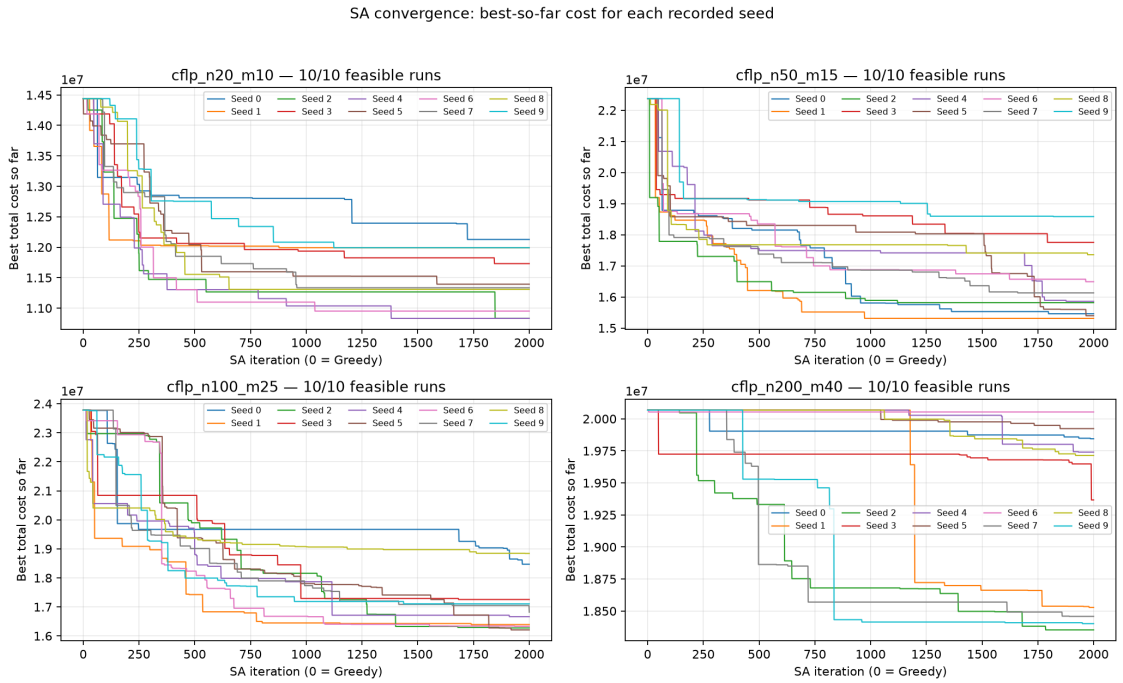


Figure 2: SA convergence for all four instances (10 seeds each). Each line shows best-so-far cost from iteration 0 (greedy starting solution) through 2,000 SA iterations.

7. Critical Analysis and Discussion

The two methods show fundamentally different characteristics reflecting the theoretical properties of exact and heuristic optimization.

Exact method strengths and limitations. The MILP solved all four instances to provable optimality (0% gap). This is the key advantage of exact methods: the LP relaxation lower bound certifies that no better solution exists. However, runtime grows sharply from 0.03s at n=20 to 39.15s at n=200 and would likely exceed the time limit for larger instances. CFLP is NP-hard, so exponential worst-case growth is expected as the Branch and Bound tree expands.

Heuristic strengths and limitations. SA scaled more gracefully in runtime and found feasible solutions for every seed on every instance. On n=20, the best run matched the MILP optimum exactly. On larger instances the optimality gap grew to 32.6% at n=200. The convergence plots (Figure 2) show most improvement occurs within the first 500 iterations, with later iterations yielding diminishing returns, suggesting the cooling schedule could be made more aggressive.

Solution variance across seeds. SA quality varies between seeds on the same instance. Standard deviations range from 491,853 at n=20 to 1,120,896 at n=50. This reflects the stochastic nature of SA: different random states lead to different local search trajectories. Running multiple seeds and reporting the best result is therefore important for reliable performance estimation.

Growing optimality gap. The fixed 2,000-iteration budget covers a smaller fraction of the search space as problem size grows, explaining the widening gap. Increasing iterations, tuning the cooling schedule, or adding larger neighbourhood moves such as multi-customer relocation could reduce the gap substantially.

Real-world interpretation. The MILP solution for n=200 placed 10 warehouses across Brazil's main regions, consistent with real logistics intuition: concentration in São Paulo state, with regional hubs for the South, Northeast, Centre-West, and Minas Gerais. This validates that the cost model and modelling assumptions produce geographically sensible results.

7.1 Future Work

Increase SA iteration count or use a restart strategy that re-initialises from the best found solution with elevated temperature.

Tune the cooling schedule adaptively: slow cooling when improvement is frequent, faster when the search stagnates.

Add larger neighbourhood moves, such as relocating all customers of one warehouse simultaneously.

Test on instances beyond n=200 to identify the point at which the MILP time limit is consistently exceeded.

Validate the cost assumptions against real Olist shipping data to improve model realism.

8. Implementation Notes

All code is implemented in Python. Notebooks were run in Google Colab; scripts are run from the repository root. The full implementation is available at: https://github.com/SajithKumara/IT-5082-OM-Assignment

scripts/preprocess.py: Builds CFLP instances from raw Olist CSV files.
scripts/cflp_common.py: Shared instance loading, validation, and evaluate() function used by both methods.
scripts/heuristic.py: Greedy construction and Simulated Annealing implementation.
scripts/run_heuristic_experiments.py: Runs 10 seeds per instance and records all results and convergence history.
scripts/compare_methods.py: Compares MILP and SA results and produces all comparison plots.
notebooks/milp_facility_location.ipynb: MILP formulation, experiment runs, and result plots.
notebooks/heuristic_facility_location.ipynb: SA implementation and experiment notebook.

PuLP is pinned to version 2.9.0 in requirements.txt to avoid API incompatibility with PuLP 4.0. The shared evaluate() function ensures both methods use identical cost computation and feasibility checking, making the comparison fair and reproducible.

9. Individual Contribution

MS26916576 – Malagala P R: Simulated Annealing implementation (heuristic.py, run_heuristic_experiments.py, compare_methods.py), SA experiment runs, convergence and comparison plots, SA results analysis.
MS26915968 – Kumara I P S: Data preprocessing (preprocess.py), MILP formulation and implementation (milp_facility_location.ipynb), MILP experiment runs and result analysis, report drafting.

10. Conclusion

This assignment applied two optimization methods to the Capacitated Facility Location Problem using real-world data from the Olist Brazilian e-commerce platform. The MILP exact method found provably optimal solutions on all four instances, with runtime growing from 0.03 seconds to 39.15 seconds as the number of customer cities increased from 20 to 200. The LP relaxation lower bound confirmed optimality in every case. The Simulated Annealing heuristic matched the optimum on the smallest instance and produced feasible solutions on all instances, but with an optimality gap growing to 32.6% at n=200.

The results clearly demonstrate the trade-off between exactness and scalability fundamental to NP-hard combinatorial optimization. Exact methods guarantee optimality

but face exponential growth. Heuristics are scalable and practical for large instances but cannot certify solution quality. For small-scale real logistics decisions MILP is the preferred choice; for large-scale deployment Simulated Annealing provides a practical, controllable solution that can be improved with further tuning of the iteration budget and cooling schedule.

References

Olist Brazilian E-Commerce Public Dataset. Kaggle. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

Cornuejols, G., Nemhauser, G.L., Wolsey, L.A. (1990). The uncapacitated facility location problem. In: Discrete Location Theory. Wiley.

Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P. (1983). Optimization by Simulated Annealing. Science, 220(4598), 671–680.

PuLP Documentation. https://coin-or.github.io/pulp/

IT5082 Optimization Methods Lecture Notes. SLIIT. Lectures 5 and 10.

Appendix: Source Code

A.1 Shared Evaluation Function (cflp_common.py)

```
def evaluate(solution, instance):
   validate_instance(instance)
   n, m = instance['n_customers'], instance['n_facilities']
   opened = set(solution)
   total_cost = sum(instance['facilities'][i]['fixed_cost'] for i in opened)
   total_cost += sum(instance['assign_cost'][i][j]
                 for j, i in enumerate(solution))
   loads = [0] * m
   for j, i in enumerate(solution):
      loads[i] += instance['customers'][j]['demand']
   return total_cost, all(loads[i] <= instance['facilities'][i]['capacity']
                    for i in opened)
```

A.2 MILP Core (milp_facility_location.ipynb)

```
# Decision variables
y = [LpVariable(f'y_{i}', cat='Binary') for i in range(m)]
x = [[LpVariable(f'x_{i}_{j}', cat='Binary') for j in range(n)]
    for i in range(m)]

# Objective: minimise opening costs + transport costs
prob += (lpSum(facilities[i]['fixed_cost'] * y[i] for i in range(m))
     + lpSum(assign_cost[i][j] * x[i][j]
          for i in range(m) for j in range(n)))

# C1: every customer served by exactly one warehouse
for j in range(n):
   prob += lpSum(x[i][j] for i in range(m)) == 1

# C2: only assign to open warehouses
for i in range(m):
   for j in range(n):
      prob += x[i][j] <= y[i]

# C3: capacity constraint
for i in range(m):
   prob += lpSum(customers[j]['demand'] * x[i][j]
            for j in range(n)) <= facilities[i]['capacity'] * y[i]
```

A.3 Simulated Annealing Core (heuristic.py)

```
# SA main loop
for iteration in range(1, iterations + 1):
    candidate = propose_neighbor(current, instance, rng)
    if candidate is not None:
        candidate_cost, _ = evaluate(candidate, instance)
        delta = candidate_cost - current_cost
        # Accept improvement always; accept worse move with Metropolis probability
        accept = delta <= 0 or math.log(rng.random()) < -delta / temperature
        if accept:
            current, current_cost = candidate, candidate_cost
            if current_cost < best_cost:
                best, best_cost = list(current), current_cost
    # Cool down
    temperature = max(temperature * cooling_rate, 1e-300)
```